



Developing AI Agents with Agno

Demo 1

Demo: Data Analysis Agent

Overview

This is a Data Analysis application built using Phidata's agent framework that allows users to upload datasets and receive structured, AI-powered summaries. The application automatically reads the file, analyzes key statistics, breaks down the metadata into chunks, embeds those using Sentence Transformers, and stores them in Pinecone for semantic retrieval. It features a conversational agent interface powered by Google Gemini and is deployed with a user-friendly UI via Streamlit.

Scenario

Data analysts, researchers, and business professionals often work with large numbers of files containing raw tabular data. Whether sourced from clients, stakeholders, or internal teams, these datasets typically lack proper documentation or summaries. This makes it difficult to quickly understand the structure, significance, and quality of the data without opening each file and manually inspecting its contents. Over time, this process becomes repetitive, error-prone, and time-consuming—especially when trying to compare newly received data with previously analyzed files. There's a need for a smart, automated assistant that can analyze files, summarize them, and remember their key features for future reference.

Problem Statement

The goal is to build an intelligent Data Analysis Agent that helps users efficiently understand and organize their datasets. The solution should allow users to upload files, automatically generate a statistical and structural summary of each file, and embed that information into a vector database for semantic similarity searches. This AI-powered system should enable users to retrieve previously uploaded datasets based on natural language queries and see how they compare with new data. By using a modular agent-based approach, the system should provide conversational interactions, streamline

dataset analysis, and support scalable, long-term knowledge retention from structured data files.

Approach for the Solution

This application follows a modular, agent-based architecture built using Phidata, Google Gemini, SentenceTransformers, Pinecone, and Streamlit. It is designed to streamline how users understand, manage, and search across tabular datasets (CSV files). The core idea is to turn raw, unstructured CSVs into analyzable, semantically searchable knowledge—accessible through a user-friendly web interface and powered by conversational AI. Below is a breakdown of the solution architecture:



Environment Setup and API Configuration

The system starts by securely loading API keys required for connecting to external services:

- **Google API Key:** Used to authenticate with the Gemini LLM (Google's generative AI).
- **Pinecone API Key:** Used to interact with the vector database for embedding-based search.

These keys are either stored in a `.env` file or set via `os.environ`.

2. Model Initialization

Two core models are initialized:

- **Gemini (LLM):** Acts as the main reasoning engine for interpreting queries and generating natural language responses.
- **SentenceTransformer:** Used to convert text (metadata summaries) into high-dimensional vector embeddings. The `all-MiniLM-L6-v2` model is selected for its speed and performance in semantic tasks.

3. Vector Database Setup (Pinecone)

The application uses Pinecone to persist and search metadata about CSVs. A robust setup is implemented to:

- Create the index (`data-insights`) if it doesn't exist.
- Handle errors gracefully (e.g., corrupted index recovery).
- Ensure all stored summaries can be searched using semantic similarity.

Each dataset is represented by multiple embedded chunks for fine-grained search performance.

4. Tooling via Agent Functions

Three key tools are implemented and registered as part of the **Phidata** Agent:

a. **describe_data(filepath: str) → str**

- Reads a CSV file and generates a summary using **pandas.DataFrame.describe()**.
- Converts the summary into a readable string format.

b. **embed_and_store(filepath: str) → str**

- Converts the summary into chunks (~300 characters).
- Uses SentenceTransformers to embed each chunk.
- Stores the vectorized chunks in Pinecone using the CSV filename and chunk ID.

c. **search_similar(query: str) → str**

- Encodes a natural language query into an embedding.
- Searches Pinecone for similar metadata chunks.
- Returns the IDs and similarity scores of the closest matches.

These tools modularize each step of the analysis and search workflow and are reusable across different user queries.

5. Agent and Workflow Construction

The tools above are encapsulated in a single **Phidata** Agent that supports conversational queries like:

- “Describe the uploaded CSV.”
- “Store this file’s structure for later.”
- “Find similar datasets to this one.”

A **Workflow** wraps this agent to define a broader interaction flow, enabling extensibility if additional agents or tasks are added later (e.g., anomaly detection or data cleaning).

6. Front-End User Interface (Streamlit)

A simple and intuitive front-end is built using Streamlit to support the following interactions:

- **File Upload:** Users can drag and drop a **.csv** file. The system saves the file locally for analysis.
- **Automated Summary Generation:** Upon upload, the agent auto-generates a statistical summary of the dataset and displays it.
- **Similarity Search:** Users can enter a natural-language query (e.g., “Find datasets with time series data”), and the system retrieves similar previously uploaded datasets.
- **CSV Download:** The user can download the generated summary in **.csv** format for documentation or reporting.

7. Conversational Flow and Multi-Modal Reasoning

The entire system operates through a conversational interface, which allows users to simply ask questions about their data. The agent interprets these questions, routes them to the appropriate tool (e.g., describe, embed, or search), and returns results. This transforms raw files into interactive, intelligent assets that the user can explore semantically.

8. Resilience and Error Handling

Special attention is given to fault tolerance:

- Index creation and deletion are wrapped in **try-except** blocks.
- Error messages are shown in the Streamlit UI when Pinecone or Gemini encounters issues.

- Fallback mechanisms try to repair corrupted or unavailable indexes..

Key Technologies Used

Component	Purpose
Phidata Agent	Manages tool orchestration and workflows
Google Gemini	Generates summaries and interprets queries
SentenceTransformers	Embeds dataset metadata
Pinecone	Stores and retrieves semantic vector data
Streamlit	Provides a user-facing web interface
Pandas	Handles CSV parsing and analysis

Project Structure:

The project is organized as follows:

- **app4.py:** Main script containing the core logic for indexing, agent creation, and query handling.
- **data.csv:** Dataset containing employee records

attrition	gender	birthday	age	staff_category	stock_options	job_title
-----------	--------	----------	-----	----------------	---------------	-----------

- **requirements.txt:** Lists dependencies (phidata, pinecone, pandas, google-generativeai, python-dotenv).
- **.env:** Stores environment variables (e.g., GOOGLE_API_KEY).

Structure of the Solution

The structure of the solution is designed to be modular, utilizing LangChain components for different tasks:

1. **Upload Interface:** Streamlit provides CSV upload functionality.
2. **Statistical Description Tool:** Generates summaries using `pandas.DataFrame.describe()`.
3. **Chunking & Embedding Tool:** Summaries are chunked by character count and embedded.
4. **Vector Store Integration:** Pinecone stores embeddings and enables similarity search.

5. **Agent with Tools:** Agent orchestrates the above tools via a user prompt.
6. **UI Output:** Streamlit interface displays the summary and allows data download.



Stepwise Solution

Step 1: Setting up the Environment

- **Objective:** Initialize environment variables, install dependencies, and set API keys.
- **Topics Covered:**
 - Environment setup
 - Using `.env` or `os.environ`

Code:

```
os.environ['GOOGLE_API_KEY'] = 'your-google-key'  
os.environ['PINECONE_API_KEY'] = 'your-pinecone-key'
```

Step 2: Uploading and Ingesting CSV

- **Objective:** Let users upload CSV files and read them using Pandas.
- **Topics Covered:**
 - Streamlit file uploader
 - CSV parsing with Pandas

Code:

```
uploaded_file = st.file_uploader("Upload a CSV", type=["csv"])  
df = pd.read_csv(uploaded_file)
```

Step 3: Generating Descriptive Stats

- **Objective:** Automatically describe the contents of the uploaded file.
- **Topics Covered:**
 - Data summarization using pandas

Code:

```
def describe_data(filepath: str) → str:  
    df = pd.read_csv(filepath)  
    return df.describe().to_string()
```

Step 4: Chunking and Embedding Metadata

- **Objective:** Split the summary into chunks and encode with **SentenceTransformer**.
- **Topics Covered:**
 - Text chunking
 - Embeddings

Code:

```
]chunks = [summary[i:i+300] for i in range(0, len(summary), 300)]  
embedding = embedding_model.encode(chunk).tolist()
```

Step 5: Storing in Pinecone

- **Objective:** Store chunk embeddings in a vector DB for later retrieval.
- **Topics Covered:**
 - Pinecone index setup
 - Vector **upsert**

Code:

```
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc.create_index(name="data-insights", dimension=384)
index.upsert(vectors)
```

Step 6: Querying for Similar Datasets

- **Objective:** Enable semantic search based on natural language queries.
- **Topics Covered:**
 - Query vector generation
 - Retrieval from Pinecone



Code:

```
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc.create_index(name="data-insights", dimension=384)
index.upsert(vectors)
```

Step 7: Assembling the Agent

- **Objective:** Use Phidata's Agent and Workflow to integrate all tools.
- **Topics Covered:**
 - Phidata agents
 - Tool orchestration

Code:

```
agent = Agent(tools=[describe_data, embed_and_store, search_similar], model=model)
workflow = Workflow(agents=[agent])
```

Step 8: Building the UI with Streamlit

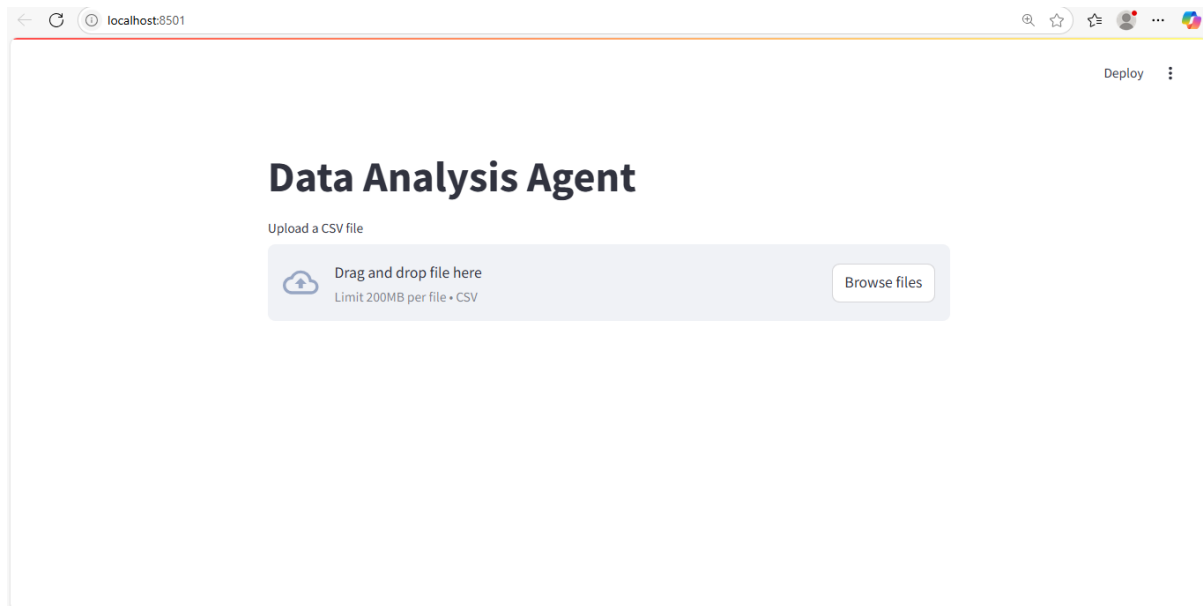
- **Objective:** Deploy a Streamlit interface for interaction.
- **Topics Covered:**
 - Streamlit components
 - User interaction flow

Code:

```
st.title("CSV Data Insights")
st.text(agent.run("Describe the dataset in uploaded_data.csv"))
```

User Interface and Output:

User Interface:



Output:

		age	employee	recent_sal	prior_sal	last_raise	commute	percent_tr	employee_id
1									
2	count	1821	1821	1821	1821	1821	1821	1821	1821
3	mean	47.4849	65.34761	124521.5	117041	7480.488	22.61669	54.50357	2E+09
4	std	7.37373	20.59141	43997.97	41595.74	6361.387	13.25926	26.66301	525.8217
5	min	35	30	50011	43975	0	0	10	2E+09
6	25%	41	47	85275	80658	1962	11	31	2E+09
7	50%	47	65	124544	116950	6318	23	54	2E+09
8	75%	54	84	162981	153402	11720	34	78	2E+09
9	max	61	100	199951	199742	25995	45	100	2E+09

Conclusion

This solution demonstrates how Phidata, Google Gemini, Pinecone, and Sentence Transformers can be combined to create a powerful, AI-first data exploration tool. The agent helps users summarize, store, and retrieve insights from files via a conversational interface — all within an easy-to-use Streamlit web app.